RUANDER
OKTATÓKÖZPONT

Tesztautomatizálás

Botlik Béla

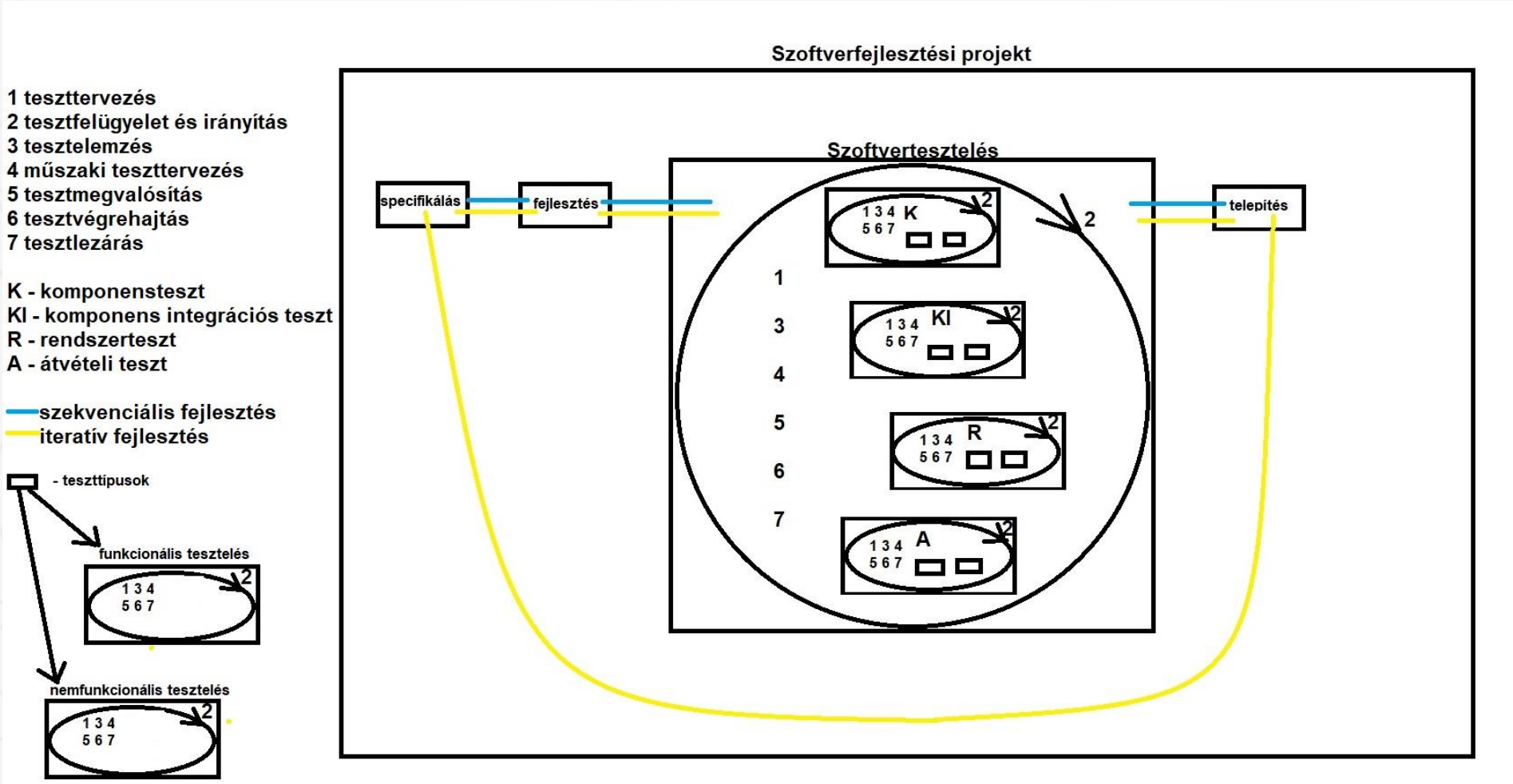
CTFL ismétlés I.

Tesztevékenység	Feladatok	Munkatermékek
Teszttervezés	A teszt céljának, tárgyának, megvalósításának meghatározása.	teszterv, tesztütemterv, kockázati nyilvántartás, belépési és kilépési feltételek
Tesztfelügyelet és -irányítás	A tesztevékenységek ellenőrzése és irányítása.	tesztjelentések, irányelvek, kockázati információk, tesztelőrehaladási jelentések
Tesztelemzés	A tesztbázis elemzése a szoftver tesztelhetőségének feltárására.	tesztfeltételek, tesztbázis hibajelentései
Műszaki teszttervezés	Tesztesetek készítése, tesztkörnyezet tervezése.	tesztesetek, lefedettségi elemek, tesztadatokra és a tesztkörnyezetre vonatkozó követelmények
Tesztmegvalósítás	Teszteljárások, tesztkészletek készítése, tesztkörnyezet megvalósítása.	teszteljárások, tesztkészletek, tesztszkriptek, tesztadatok, végrehajtási ütemterv, tesztkörnyezet
Tesztvégrehajtás	A tesztek futtatása a tesztelési ütemterv szerint.	tesztnaplók, hibajelentések
Tesztlezárás	Tesztjelentések készítése.	összefoglaló tesztjelentés, változtatáskérés

CTFL ismétlés II.

	tesztbázis	tesztcél	teszt tárgya	hibák	megközelítés
komponensteszt	tervek, kód, adatmodell	komp. verifikálás, hibák feltárása, bizalom kivívása	komponens, osztály, adatstruktúra	hibás funkcionalitás, programozói hibák	Kódközeli, tipikusan a fejlesztők végzik.
komponensintegrációs teszt	tervezési diagramok, interfész leírások	kockázat csökkentés, interfész verifikálás, hibák feltárása	interfészek	helytelen/hiányos adatok, hívások helytelen sorrendje, adatok félreértelmezése	Funkcionalitás helyett a kommunikációra fókuszál, a fejlesztők végzik.
rendszerteszt	rendszer/szoftver-követelmények	rendszer verifikálás, bizalom kivívása	alkalmazás, op.rendszer, rendszer	helytelen viselkedés, helytelen vezérlés	Végponttól végpontig terjedő tesztek, független tesztelők is végezhetik.
rendszerintegrációs teszt	rendszerterv, használati esetek	kockázat csökkentés, interfész verifikálás, hibák feltárása	alrendszerek, API-k, külső szolgáltatások	rendszerek közötti kommunikáció hibái, átadott adatok félreértelmezése	Funkcionalitás helyett a kommunikációra fókuszál, a tesztelők végzik.
elfogadási teszt	rendszer/szoftver-követelmények	rendszer validálás, kiadásra való felkészültség igazolása	rendszer	követelményeknek nem megfelelés	A tesztek felelősei üzleti felhasználók, vevők.

CTFL ismétlés III.



Bevezetés és célok a tesztautomatizálás terén (I. fejezet)

- A tesztautomatizálás célja
- A tesztautomatizálás előnyei
- A tesztautomatizálás hátrányai
- A tesztautomatizálás korlátai
- Tesztautomatizálás különböző fejlesztési életciklusmodellekben
- Teszteszköz választás

Tesztautomatizálás

- A tesztautomatizálás tevékenységei:
  - célzott szoftvereszközök használata tesztkészletek előkészítéséhez és kezeléséhez a tesztvégrehajtás érdekében;
  - tesztek automatizált végrehajtása;
  - az aktuális és elvárt eredmények összehasonlítása.
- A tesztautomatizálás széleskörű szoftverterületeket fed le és sokféle szoftverre vonatkozhat
  - UI SUT; SUT UI nélkül; mobilappok; hálózati protokollok és kapcsolatok

A tesztautomatizálás előnyei, hátrányai és korlátai

Előnyök	Hátrányok	Korlátok
Több teszt futtatása	Kezdeti befektetés szükséges	Nem minden teszt automatizálható
Manuálisan végre nem hajtható tesztek futtatása	A tesztautomatizálási megoldás fejlesztése és karbantartása időigényes lehet	Csak azt tudja tesztelni, amire programozták
Összetettebb tesztek elvégzése	Esetleges plusz költségek	Teszt-orákulum hitelesítést igényel
Tesztelési erőforrások hatékony kihasználása	Kevésbé alkalmazkodó a rendszer változásaihoz	Csak géppel értelmezhető teszteredményeket tud ellenőrizni
Kevesebb emberi hiba	Új hibákat vezethet be	
Gyorsabb végrehajtás	Világos tesztcélokra van szükség	
Gyors visszajelzés a minőségről		
Javítja a rendszer megbízhatóságát		
Következetes tesztvégrehajtás		

Tesztautomatizálás és fejlesztési modellek

- A különböző fejlesztési modellek más jellemzőkkel bírnak a tesztautomatizálás szempontjából:
  - Vízesés modell: a tesztautomatizálás a megvalósítással párhuzamosan vagy azt követően zajlik.
  - V-modell: minden fejlesztési szinthez tartozik tesztelési szint.
  - Agilis fejlesztés: gyakori automatizált tesztvégrehajtás jellemzi.
- A legmegfelelőbb teszteszköz kiválasztásához a tesztelt rendszert (SUT) és a csapat összetételét kell kiértékelni.

Felkészülés a tesztautomatizálásra (II. fejezet)

- Tesztelhetőség és konfiguráció
- Tesztautomatizálás végrehajtási környezetei
- A tesztelendő rendszer elemzése
- Eszközértékelés

Tesztelhetőség és konfigurációs megoldások

- Az automatizálásra való felkészülés első lépése a tesztelhetőség meghatározása.
- A jobb tesztelhetőség konfigurációs beállításokat igénylő megoldásai:
  - hozzáférhetőségi azonosítók;
  - rendszerkörnyezeti változók;
  - telepítési változók.
- A tesztelhetőség tervezésének aspektusai:
  - megfigyelhetőség;
  - irányíthatóság;
  - architektúra átláthatósága.

Tesztautomatizálás különböző környezetben

- Automatizált tesztek végrehajtási környezetei:
  - lokális fejlesztési környezet: komponensteszt; fehérdobozos teszt
  - build környezet: CI/CD folyamat; komponensintegrációs teszt
  - integrációs környezet: feketedobozos és rendszer(integrációs) teszt
  - validációs környezet: elfogadási tesztelés
  - működési környezet: speciális tesztek (canary release, blue/green deployment, A/B teszt)

A tesztelendő rendszer elemzése

- Egy sikeres tesztautomatizálási megoldás (TAS) létrehozásának előfeltétele a tesztelendő rendszer (SUT) elemzése.
- A tesztautomatizálási megközelítés és architektúra követelményeinek szempontjai:
  - tesztttevékenységek;
  - tesztszintek;
  - teszttípusok;
  - tesztszerepkörök;
  - szoftvertermékek;
  - SUT és TAS kompatibilitás;
  - tesztadatok elérhetősége és minősége;
  - harmadik féltől származó alkalmazások emulálása.

Feladat I.

- Elemezze a tesztelt rendszert (SUT) és keressen tesztautomatizálási megoldást!
  - A vizsgált rendszer egy webáruház, amely lehetővé teszi a felhasználók számára:
    - Regisztrációt és bejelentkezést
    - Termékek keresését és megtekintését
    - Kosárba helyezést és vásárlást
    - Fizetést PayPal integrációval
  - A rendszer többféle technológiai rétegből áll:
    - Frontend (React)
    - Backend (Java – Spring Boot)
    - Adatbázis (MySql)
    - Külső API (PayPal)

Az eszközértékelés eredményei

- A rendszer elemzéséből táblázatos formában összeáll a használható eszközök köre, legfőbb tulajdonságaikkal:
  - technológia és IDE;
  - konfigurálhatóság;
  - tesztadatok kezelhetősége;
  - teszttípusonkénti eltérések;
  - riportálási képesség;
  - más eszközökkel való integrálhatóság;
  - az architektúra bővítésének lehetősége.

Feladat II.

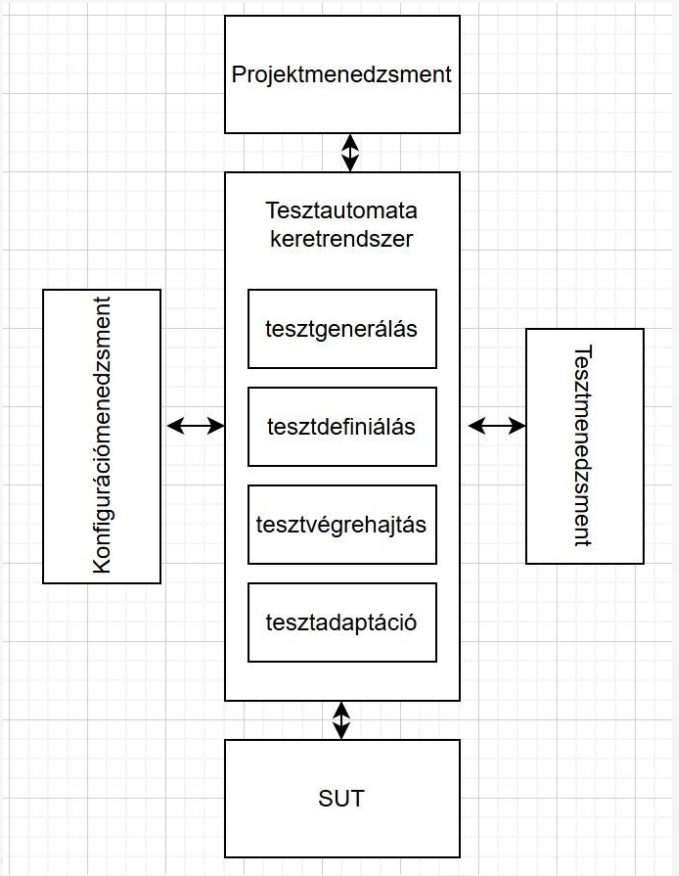
- Az előző feladat webáruházának PayPal API teszteléséhez első körben 3 eszköz merült fel:
  - Postman - Grafikus felhasználói felülettel rendelkező API tesztelő
  - RestAssured - Java alapú API tesztelési könyvtár fejlesztőközpontú megközelítéssel
  - Karate - BDD megközelítésű API tesztelő keretrendszer
- Töltse ki az eszközértékelés eredményeit összehasonlító táblázatot!

Tesztautomatizálási architektúra (III. fejezet)

- Általános tesztautomatizálási architektúra
- Tesztautomatizálási megoldás tervezése
- Tesztautomatizálási keretrendszer (TAF)
- Tesztautomatizálási megközelítések
- Tervezési alapelvek és tervezési minták a tesztautomatizálásban

A tesztautomatizálási architektúra és fő képességei

- A tesztautomatizálási architektúra interfészei:
  - SUT interfész;
  - Projektmenedzsment interfész;
  - Tesztmenedzsment interfész;
  - Konfigurációmenedzsment interfész.
- Az automatizálási keretrendszer képességei:
  - Tesztgenerálás;
  - Tesztdefiniálás;
  - Tesztvégrehajtás;
  - Tesztadaptáció.



Tesztautomatizálási megoldás tervezése

- A tesztautomatizálási megoldás (TAS) tervezése a SUT követelményei és a szükséges eszközök alapján kerül megvalósításra, melynek elemei:
  - Tesztautomata eszközök és eszközspecifikus könyvtárak kiválasztása
  - Beépülő modulok és/vagy komponensek fejlesztése
  - Kapcsolati és interfész követelmények azonosítása
  - Kapcsolódás tesztmenedzsment és hibakezelő eszközökhöz
  - Verziókezelési rendszer és adattárak használata

Tesztautomatizálási keretrendszer

- A TAS technikai alapja a tesztautomatizálási keretrendszer (TAF).

Elemei	Jellemzői	Rétegei
tesztellenőrző rendszer	struktúráltság	tesztszkriptek
tesztkönyvtárak	automatizálási hatékonyság	üzleti logika
tesztszkriptek	skálázhatóság	alapkódkönyvtárak
tesztkészletek		

Feladat III.

- Alkalmazza a tesztautomatizálási keretrendszer rétegezését egy webalkalmazás funkcionális teszteléséhez (belépési funkció)!

Tesztautomatizálási megközelítések I.

- Számos megközelítés létezik a tesztautomatizálás terén, melyek különböző körülmények között alkalmazhatók:

Megközelítés	Főbb jellemzők	Előnyök	Hátrányok
Rögzítés/lejátszás	A manuális tesztelési műveletek elektronikus rögzítésén és visszajátszásán alapul.	Kezdetben könnyen beállítható és használható.	Fenntartása/továbbfejlesztése nehézkes; SUT elérhető kell legyen; Ritkán és kismértékben változó SUT esetén ésszerű; SUT-verziófüggő; A tesztesetek rögzítése időigényes lehet.
Lineáris szkriptelés	A rögzítés/lejátszás tesztjeit felhasználó programozási tevékenység	Gyors és egyszerű; A tesztszkriptek módosíthatók.	Fenntartása/továbbfejlesztése nehézkes; SUT elérhető kell legyen; Ritkán és kismértékben változó SUT esetén ésszerű; Némi programozási ismeret szükséges.

Tesztautomatizálási megközelítések II.

Megközelítés	Főbb jellemzők	Előnyök	Hátrányok
Struktúrált szkriptelés	Blokkokba, könyvtárakba szervezett szkriptek	újrafelhasználható; karbantartható; rugalmas; az üzleti logika elkülöníthető.	programozási ismeret szükséges; nagyobb kezdeti befektetés; komplexitás.
Adatalapú tesztelés (DDT)	Struktúrált szkriptre épülő, adatokkal ellátott szkriptek	tesztesetek egyszerű bővítése; tesztköltségek csökkenése; tesztelemzők nagyfokú szabadsága.	adatkezelési kihívások
Kulcsszóvezérelt tesztelés (KDT)	Gyakran DDT-re épülő, kulcsszavak és tesztadatok alapján származtatott lépések listája vagy táblázata	teszt -és üzleti elemzők részvétele a tesztesetek létrehozásában; manuális tesztekre is alkalmazható; kevésbé függ a fejlesztktől.	A kulcsszavak implementálása és karbantartása összetett feladat; Kis rendszerek esetén jelentős erőforrás igénye van.

Tesztautomatizálási megközelítések III.

Megközelítés	Főbb jellemzők	Előnyök	Hátrányok
TDD (tesztvezérelt fejlesztés)	A tesztek a fejlesztés részeként és elsőként kerülnek meghatározásra	egyszerű teszteset fejlesztés; jobb kódminőség, - tesztelhetőség, - kódlefedettség elérés; csökkenti a hibák tovább terjedését; javuló kommunikáció; gyors kilépési kritérium elérés.	TDD-re áttérés időigényes;
BDD (viselkedésvezérelt fejlesztés)	Természetes nyelvformátum használatával kialakított elfogadási kritériumok, amik egyúttal tesztesetek is	javuló kommunikáció; BDD forgatókönyvek tesztesetként biztosítják a specifikáció lefedettségét; többféle teszttípus előállításában használható a tesztpiramis különböző szintjein.	további teszteseteket kell meghatározni; üzleti elemzők és fejlesztők kihagyása; tesztlépések implementálása bonyolult; emiatt nehézkes és költséges hibakeresés.

Feladat IV.

- Határozzon meg különböző tesztautomatizálási megközelítéseket egy e-kereskedelmi webáruház kosár és fizetés folyamat teszteléséhez!

Tervezési alapelvek és tervezési minták a tesztautomatizálásban

- A szoftverfejlesztő mellett a tesztautomatizálási mérnöknek is ismernie kell a tervezési alapelveket és mintákat:

OOP alapelvek	SOLID alapelvek	Tervezési minták
absztrakció	egyért felelősség	Facade pattern
egységbezárás	nyitottság és zártság	Singleton pattern
adatrejtés	Liskov helyettesítés	POM (Page Object Model)
öröklődés	interfész helyettesítés	Folyamat-alapú modell (FMP)
polimorfizmus	függőség inverzió	

Feladat V.

- Adjon meg néhány tervezési mintát -a tervezési alapelvek betartásával- egy e-kereskedelmi webáruház termék kereső és kosárba helyezés folyamatához!